

Laboratorio di Basi di Dati: Progetto "Soccorso"

Gruppo di lavoro

Matricola	Nome	Cognome	Contributo al progetto
298633	Alessandro	Di Marzio	Tutto (in particolare query)
291914	Isaac	Fano	Tutto (in particolare interfaccia grafica)

Data di consegna del progetto: 30/06/2026

1. Analisi dei Requisiti e Scelte Progettuali

1.1 Riassunto dei Requisiti

Il sistema "Soccorso" ha l'obiettivo di gestire la ricezione di richieste di emergenza e il coordinamento delle relative missioni di intervento. Il dominio prevede la presenza di utenti divisi in amministratori (che coordinano il sistema e gestiscono anagrafiche, mezzi e materiali) e operatori (impiegati fisicamente sul campo). Ogni operatore possiede specifiche patenti e abilità, necessarie per l'impiego di particolari veicoli (ambulanze, autopompe, ecc.) e materiali (defibrillatori, estintori, ecc.). Le richieste, inoltrate dai cittadini, seguono un ciclo di vita specifico (da inviate a chiuse) e sfociano in missioni, in cui vengono allocate squadre (operatori con ruoli distinti, tra cui almeno un caposquadra), mezzi e risorse materiali.

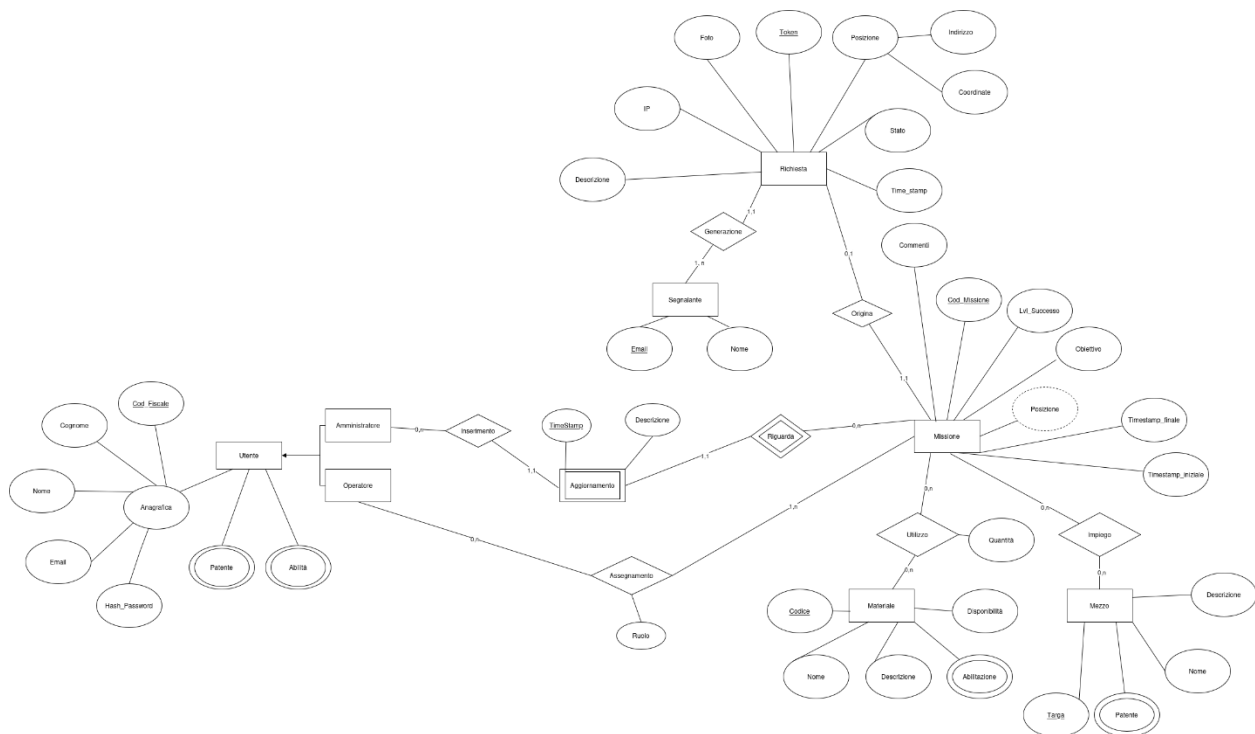
1.2 Scelte Progettuali rispetto alla Specifica

Prima di procedere alla modellazione vera e propria, sono state prese delle decisioni per disambiguare e raffinare quanto descritto nella traccia del committente:

- **Gestione degli Utenti e Gerarchie:** La traccia distingue tra amministratori e operatori. Poiché entrambe le categorie condividono gli stessi attributi fondamentali (anagrafica, credenziali, ecc.), si è deciso di non creare entità separate o gerarchie complesse in fase iniziale, ma di trattare il dominio tramite un concetto unificato di "Utente". Inoltre per quanto riguarda l'operatore che può essere anche caposquadra, in merito alla relazione assegnamento troviamo l'attributo ruolo, che ci permette di differenziare un operatore semplice da un caposquadra.
- **Ciclo di vita della Richiesta:** Per modellare i passaggi di stato (inviata, attiva, in corso, chiusa, annullata), si è deciso di tracciare l'evoluzione direttamente nell'entità della richiesta tramite un attributo di stato. La validazione per "attivare" la richiesta viene gestita tramite l'emissione e il controllo di un token univoco, oltre alla registrazione dell'indirizzo IP per mitigare gli abusi.

- **Distinzione tra Assegnamento, Impiego e Utilizzo:** Per tracciare storicamente le missioni è stato stabilito che una missione aggrega risorse differenti. Si sono concettualizzate azioni separate: gli operatori vengono "assegnati" (con un ruolo specifico, es. caposquadra), i mezzi vengono "impiegati" e i materiali vengono "utilizzati" (specificando la quantità prelevata dal magazzino, se quest'ultima risulta disponibile, ovvero ci sono abbastanza materiali di quel tipo disponibili in magazzino).

2. Progettazione Concettuale



Nel modello concettuale, sono state individuate le entità principali: **Utente, Richiesta, Missione, Mezzo e Materiale**, oltre che l'entità debole **Aggiornamento** e un'entità strettamente legata alla richiesta, ovvero il **segnalante**. Le relazioni cardine ruotano attorno alla Missione, che funge da aggregatore per Utenti, Mezzi e Materiali. Si è scelto di inserire attributi sulle relazioni (come il *Ruolo* nell'assegnamento della squadra e la *Quantità* nell'uso dei materiali) in quanto queste informazioni esistono solo in virtù dell'associazione tra le risorse e lo specifico intervento. L'entità debole Aggiornamento è stata decisa tale, poiché logicamente un aggiornamento vive in funzione di una missione e non può essere visto come un'entità a sé stante. Gli utenti sono stati modellati, oltre che con un'anagrafica ben determinata (contenente anche un'hash password per motivi di sicurezza), con due attributi composti: *Patente* (poiché un singolo utente può essere abilitato a portare anche più tipi di veicoli richiedenti diverse patenti) e *Abilità* (un utente può possedere molte abilità, intese come mansioni/capacità nel relativo campo di occupazione; es il medico sa rianimare). Questi attributi servono, come vedremo

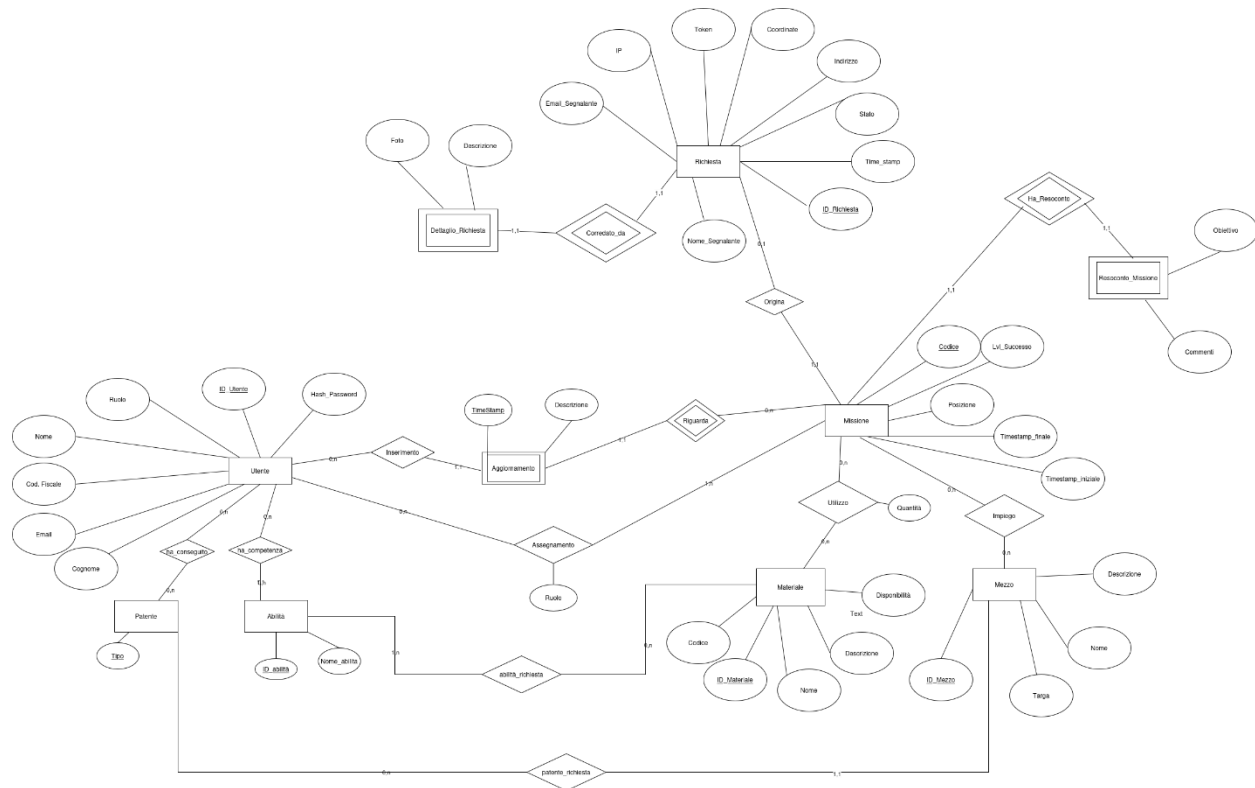
dopo, a creare un ponte con Materiale e Mezzo. Infine possiamo dire che un determinato segnalante potrà generare una richiesta che passerà determinati stati, tramite validazione con token (inviata → attiva), creazione missione (attiva → in corso), chiusura (in corso → chiusa). Inoltre in rari casi la richiesta potrebbe essere anche rifiutata e quindi annullata. La missione, come già detto avrà una squadra composta da un caposquadra e da 0 o più operatori semplici, da mezzi e materiali (usati da operatori abilitati ad essi). La missione inoltre gode di Timestamp finale e iniziale per poter anche generare uno storico, livello di successo, commenti e obiettivo. Infine possiamo notare che è presente un attributo derivabile, ovvero posizione in missione che viene determinato andando a prendere l'indirizzo della richiesta.

3. Formalizzazione dei vincoli non esprimibili nel modello ER

Dall'analisi di dominio sono emersi requisiti operativi stringenti che non trovano rappresentazione unicamente grafica nel diagramma Entità-Relazione:

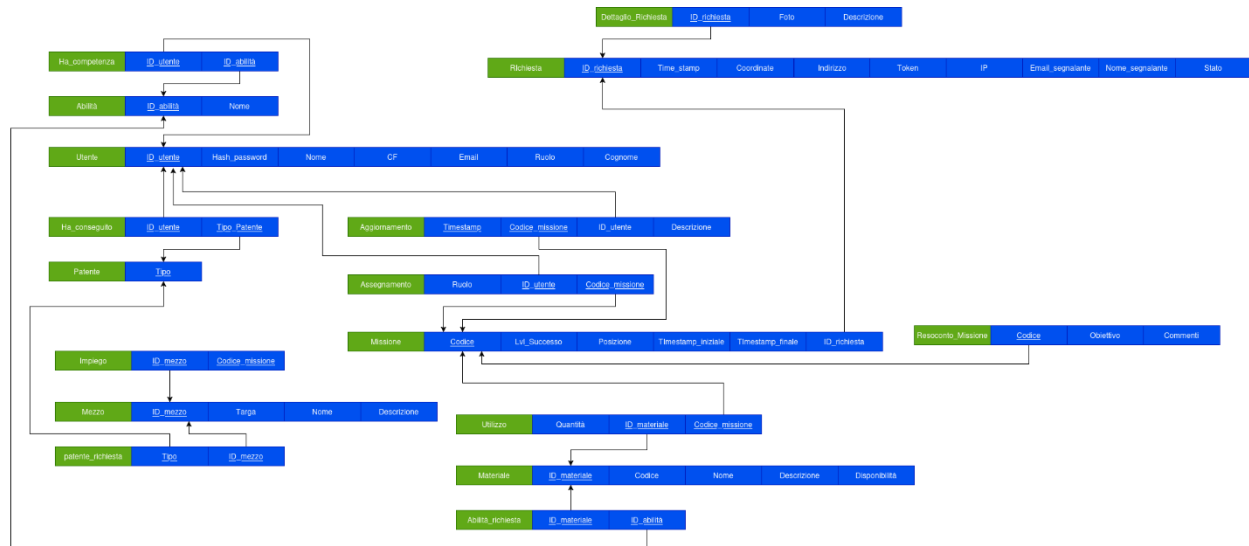
- **V1 (Disponibilità Operatori):** Un operatore non può essere assegnato a una nuova missione se risulta già coinvolto in una missione attualmente "in corso" (ovvero priva di data di conclusione).
- **V2 (Disponibilità Mezzi e Materiali):** Analogamente al personale, un mezzo di soccorso o un determinato materiale impegnato sul campo non può essere usato per una richiesta parallela finché non rientra.
- **V3 (Giacenza Materiali):** In fase di allocazione dei materiali per una missione, la quantità richiesta non deve mai superare la disponibilità effettiva residua nel magazzino.
- **V4 (Consistenza Temporale):** La marca temporale (timestamp) di chiusura di una missione deve essere necessariamente successiva o coincidente a quella di apertura.
- **V5 (Integrità Squadra):** Una missione non può essere avviata se non è stato assegnato alla squadra almeno un membro avente il ruolo di "caposquadra" a garanzia del coordinamento.

4. Progettazione Logica: Ristrutturazione ed ottimizzazione del modello ER



Nella transizione verso il modello relazionale, l'ER iniziale ha subito delle ristrutturazioni essenziali. Si è evitato di creare partizionamenti per entità che avrebbero generato frammentazione (es. Amministratore vs Operatore), implementando un attributo discriminante (**Ruolo**) all'interno dell'entità **Utente**. Abbiamo deciso di accorpare i figli al padre poiché tranne per le due relazioni **Inserimento** e **Assegnamento** le due entità risultavano uguali con gli stessi attributi, quindi per non appesantire il diagramma abbiamo deciso di inserire un attributo discriminante invece che creare due entità con un po' di attributi che avrebbero potuto pesare. Inoltre, elementi concettuali massicci come la **Richiesta** o **Missione** sono stati sciolti (partizionamento verticale) estraendo il **Dettaglio Richiesta** e il **Resoconto Missione**, ottimizzando così lo spazio e le performance di lettura per le richieste base, separandole da blob pesanti (come eventuali foto) e non usualmente richiesti a differenza dei dati principali. Infine abbiamo inglobato l'entità segnalante in richiesta, poiché come detto prima non aveva senso avere una tabella in più per una piccola entità che non avesse neanche senso di "vivere" da sola. Infine possiamo dire di aver esploso gli attributi multipli di utente, trasformandoli in entità proprie e collegandoli con le dovute "controparti" in mezzi e materiali, di aver creato molte nuove chiavi basate su id (i quali saranno definiti auto incrementali) per maggiore efficienza. Un'altra ottimizzazione è stata quella di volere la ridondanza dell'attributo posizione in missione per una maggiore efficienza a livello di query (poiché richiesta) invece di "risalire" ogni volta all'indirizzo della richiesta.

5. Traduzione del modello ER nel modello relazionale



Lo schema logico relazionale derivato vede la definizione delle chiavi primarie (es. l'ID auto-incrementale per Utente, Richiesta, Mezzo, Materiale) e delle chiavi esterne per collegare i dati storici. Le associazioni sono state tradotte in tabelle ponte. Ad esempio, la tabella **assegnamento** risolve la relazione M:N tra utente e missione, acquisendo come chiavi esterne (e chiave primaria composta) i codici identificativi dell'operatore e della missione, e ospitando l'attributo relativo al ruolo in missione.

6. Progettazione Fisica: Implementazione del modello relazionale e dei vincoli

La base di dati fisica è stata creata su DBMS MySQL prestando grande attenzione al partizionamento logico tramite vincoli FOREIGN KEY, corredati da logiche di propagazione ON DELETE CASCADE e ON UPDATE CASCADE dove la sussistenza dei record dipendenti era legata all'esistenza del padre (es. i dettagli di una richiesta). Per le storicizzazioni (es. utenti assegnati a missioni concluse), si è preferita l'azione RESTRICT o NO ACTION in cancellazione, per evitare di alterare o perdere i report storici degli interventi a seguito della rimozione di un utente o mezzo.

L'implementazione dei vincoli complessi (descritti nella sezione 3) ha richiesto l'uso di programmazione lato database (SQL procedurale). Nello specifico:

- I vincoli di consistenza temporale e quantitativa sono gestiti tramite clausole CHECK in fase di definizione tabella.
- I controlli operativi di sovrapposizione (**V1**, **V2** e **V3**) sono attuati mediante **Trigger** pre-inserimento. Ogni tentativo di inserire in squadra un utente o un mezzo in uso, o di consumare risorse esaurite, intercetta il dato, effettua un conteggio sui record aperti e genera un'eccezione bloccante con SIGNAL SQLSTATE.
- L'avvio e la chiusura della missione e il vincolo del caposquadra (**V5**) sono gestiti

incapsulando la logica in robuste **Stored Procedure**. Queste garantiscono la consistenza transazionale: l'avvio elabora e smista stringhe di input per popolare massivamente mezzi e materiali in un solo colpo; la chiusura aggiorna i timestamp e marchiare la richiesta come soddisfatta.

7. Implementazione delle funzionalità richieste

All'interno dei file del progetto si possono trovare tre file SQL contenenti rispettivamente la Struttura, con procedure e signal del sistema, una lista di esempi di query già con campi determinati (non necessariamente da eseguire poiché presenti query parametriche nell'interfaccia a disposizione) e un file usato per un popolamento iniziale del database.

Interfaccia verso il database

Per interagire con il database in modo intuitivo e validare il corretto funzionamento delle query, è stata sviluppata un'interfaccia grafica (GUI) desktop in linguaggio **Java**, basata sulla libreria standard **Swing**. Il progetto è gestito tramite **Maven** e comunica con il DBMS utilizzando **JDBC**.

Architettura e Tecnologie utilizzate: L'applicativo segue una netta separazione delle responsabilità (ispirata al pattern MVC):

- **Connection Pooling (ConnessioneDB.java):** Per ottimizzare le performance e la gestione delle risorse, le connessioni al database sono gestite tramite un *Connection Pool* utilizzando la libreria Apache Commons DBCP2.
- **Logica di Business (Query_Soccorso.java):** Centralizza tutta l'interazione col database. Esegue le interrogazioni (mappando i ResultSet in record Java TableData), gestisce le transazioni (es. inserimento richiesta e dettaglio con commit manuale) e invoca le *Stored Procedure* (avvia_missione e chiudi_missione).
- **Presentazione (Finestra.java e TextPresenter.java):** Fornisce una GUI divisa in schede (JTabbedPane). La parte superiore raccoglie gli input dell'utente, mentre la parte inferiore mostra lo stato del database in tempo reale e una console per l'output testuale formattato e allineato delle query.

Funzionalità invocabili dall'interfaccia: L'interfaccia copre la totalità delle operazioni richieste dalla specifica, suddivise per tab:

- **Transazioni e Modifiche (Query 1-3):** Inserimento di una nuova richiesta (con supporto all'upload di un'immagine BLOB dal file system locale), avvio di una nuova missione (Query 2) e chiusura di una missione in corso (Query 3).
- **Interrogazioni Analitiche (Query 4-13):** Esecuzione parametrica di tutte le statistiche richieste (operatori liberi, conteggio missioni, tempi medi annuali e per caposquadra, ore di utilizzo materiali, operatori coinvolti in missioni critiche, ecc.).
- **Funzioni di supporto (Anteprime):** Sono stati inseriti dei pulsanti di "Help/Anteprima" per visualizzare rapidamente gli id e i dati di operatori, mezzi, materiali e missioni, agevolando così l'inserimento dei parametri per testare le query principali.

